

第5章: Supabase セットアップとデータベース設計

この章では、**Supabase**（スーパーベース）というサービスを使って、アプリのデータを保存する**データベース**（Database：データを永続的に保管する仕組み。アプリを閉じててもデータが消えない）を準備します。

この章で学ぶこと

- **データベースとは何か** — プログラムのデータを永続的に保存する「倉庫」
- **テーブル設計** — データをどのような「表」の形で保存するか考える作業
- **SQL の基本** — データベースに命令を出すための言語（SELECT, INSERT, UPDATE, DELETE）
- **RLS（Row Level Security）** — 「誰がどのデータにアクセスできるか」を制御するセキュリティ機能
- **Supabase クライアント** — JavaScriptのコードからSupabaseに接続する方法

データベースはなぜ必要？ 変数や配列にデータを保持しても、ブラウザを閉じれば消えてしまいます。データベースはハードディスクにデータを保存するので、アプリを再起動してもデータが残ります。SNSの投稿、ECサイトの商品情報、そしてこのアプリの書籍情報も、すべてデータベースに保存されています。

目次

0. 前提知識: データベースとSQLの超基礎
1. Supabase とは
2. アカウント作成とプロジェクトセットアップ
3. データベース設計の基礎
4. 書籍管理アプリのデータベース設計
5. Row Level Security (RLS)
6. Supabase Client のセットアップ
7. Supabase の基本操作（CRUD）
8. テストデータの投入
9. トラブルシューティング

0. 前提知識: データベースとSQLの超基礎

Supabase の中身は **PostgreSQL**（ポストグレスキューエル）という有名なデータベースです。本格的に使い始める前に、データベース・テーブル・SQL のいろはを押さえておきましょう。

0.1 データベースとは何か

「データを永続的に・大量に・整理して保管しておく仕組み」がデータベース（DB）です。Excelで言えば「複数のシートを持つ巨大な台帳」のイメージです。本書で使う PostgreSQL は、最も広く使われているリレーショナルデータベース（RDB）の一つです。

0.2 テーブル・カラム・レコード

RDB では、データを **テーブル（表）** に保存します。

booksテーブル				
id	title	author	status	← カラム（列）
1	リーダブルコード	Boswell	done	← レコード（行）
2	プロを目指す人～	鈴木僚太	reading	
3	達人プログラマー	Thomas	unread	

用語	意味	Excelで言うと
テーブル	データの表そのもの	シート1枚
カラム（列）	縦の項目（id, title, author...）	A列、B列、C列...
レコード（行）	横の1件分のデータ	1行目、2行目...
主キー（PK）	レコードを一意に区別する値（普通は id ）	通し番号

0.3 SQL の超ミニマム入門

データベースを操作する言語が SQL（Structured Query Language）です。よく使う4つだけ先に覚えましょう。

SELECT — データを取り出す

```
SELECT title, author FROM books;
```

▼ 実行結果:

title	author
リーダブルコード	Boswell
プロを目指す人～	鈴木僚太
達人プログラマー	Thomas

条件付きで取り出すこともできます。

```
SELECT * FROM books WHERE status = 'reading';
```

▼ 実行結果（statusが'reading'の行だけ）：

id	title	author	status
2	プロを目指す人～	鈴木僚太	reading

INSERT — データを新しく追加する

```
INSERT INTO books (title, author, status) VALUES ('SQL入門', 'ミック', 'unread');
```

▼ 実行結果: **INSERT 0 1**（1件追加された）

実行後にテーブルを SELECT すると、新しい行が増えています。

UPDATE — データを書き換える

```
UPDATE books SET status = 'done' WHERE id = 2;
```

▼ 実行結果: **UPDATE 1**（1件更新された）。id=2 のレコードの status が **'reading'** から **'done'** に変わります。

WHERE を必ず付ける! **WHERE** を忘れると全レコードが書き換わってしまいます。常に「どの行を対象にするか」を明示しましょう。

DELETE — データを消す

```
DELETE FROM books WHERE id = 3;
```

▼ 実行結果: **DELETE 1**（1件削除された）。

これも **WHERE 必須!** 忘れると全件削除です。バックアップ無しで実行すると地獄を見ます。

0.4 CRUDという言葉

上の4つの操作を頭文字でまとめて **CRUD**（クラッド）と呼びます。

| C | Create | INSERT | データを作る || R | Read | SELECT | データを読む || U | Update | UPDATE | データを更新する |
| D | Delete | DELETE | データを削除する |

ほぼすべてのアプリは、CRUD のどれか（または組み合わせ）で動いています。本書の書籍管理アプリも、CRUD の練習が目的です。

0.5 本書での書き方: SQL 直接書かない

本書では、これらの SQL を直接書く機会は少なめです。代わりに Supabase の JavaScript クライアントを使い、TypeScript のコードで操作します。

```
// SQL の SELECT * FROM books と同じ意味
const { data } = await supabase.from("books").select("*");
console.log(data);
// ▼ data の中身 (実行結果のイメージ)
// [
//   { id: 1, title: "リーダブルコード", author: "Boswell", status: "done" },
//   { id: 2, title: "プロを目指す人へ", author: "鈴木僚太", status: "reading" },
//   ...
// ]
```

JavaScript のコードと SQL の対応関係は次のとおり。

操作	Supabase クライアント	SQL
読む	<code>.from("books").select("*")</code>	<code>SELECT * FROM books</code>
追加	<code>.from("books").insert({ title: "...", })</code>	<code>INSERT INTO books (...)</code>
更新	<code>.from("books").update({ status: "done" }).eq("id", 2)</code>	<code>UPDATE books SET status='done' WHERE id=2</code>
削除	<code>.from("books").delete().eq("id", 3)</code>	<code>DELETE FROM books WHERE id=3</code>

要点: 「SQLは知らなくても書ける」けど「SQLを知っていると見通しがよくなる」。SupabaseにはSQLエディタ画面があるので、迷ったらそこで生のSQLを書いて確認できます。

1. Supabase とは

1.1 BaaS (Backend as a Service) の概念

Web アプリケーションを作るとき、通常は「フロントエンド（画面）」と「バックエンド（サーバー・データベース）」の両方を構築する必要があります。

従来の開発フロー:

1. データベースサーバーを用意する（PostgreSQL、MySQL など）
2. API サーバーを構築する（Express、Django、Rails など）
3. 認証の仕組みを実装する
4. ファイルストレージを用意する
5. これらすべてをデプロイ・運用する

これは初心者にとって非常に大きなハードルです。学ぶべき技術が多すぎて、アプリの本質的な部分に集中できません。

BaaS（Backend as a Service） は、このバックエンド部分をまるごとクラウドサービスとして提供してくれるものです。つまり、データベース・認証・ストレージ・API といったバックエンドの機能を、自分でサーバーを構築することなく利用できます。

従来の開発：

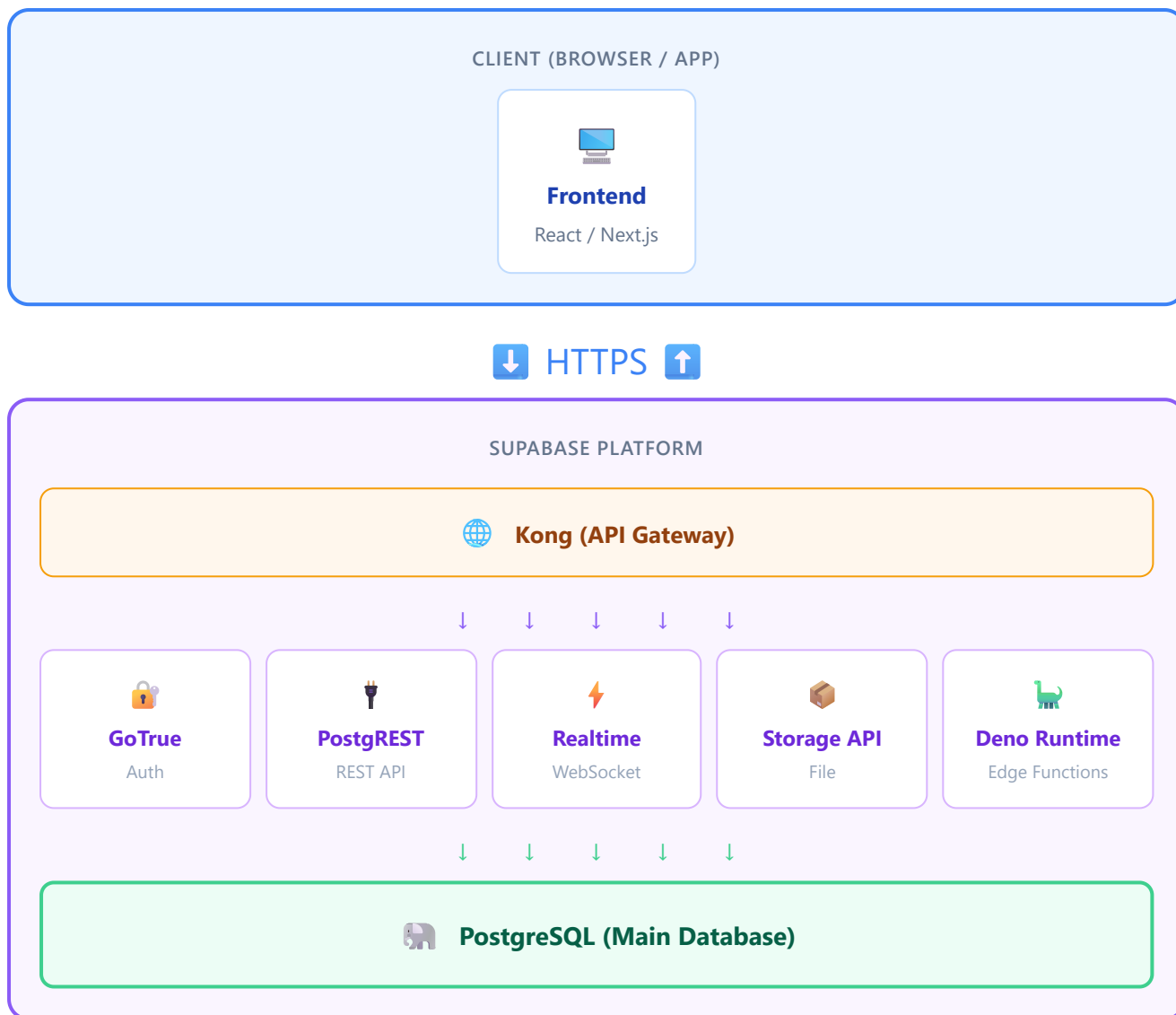
フロントエンド → 自作 API サーバー → 自作データベース → 自作認証 → 自作ストレージ
（すべて自分で構築・運用）

BaaS を使った開発：

フロントエンド → BaaS（Supabase）
（バックエンドはすべて Supabase が提供）

1.2 Supabase の全体アーキテクチャ

Supabase がどのような構成になっているか、全体像を見てみましょう。



ポイント: Supabase はオープンソースのツールを組み合わせて構築されています。PostgreSQL を中核として、PostgREST（自動 API 生成）、GoTrue（認証）、Realtime（リアルタイム通信）などが連携しています。

1.3 Firebase との比較

BaaS の中で最も有名なのは Google の **Firebase** です。Supabase は「Firebase のオープンソース代替」を目指して作られました。両者を比較してみましょう。

項目	Supabase	Firebase
データベース	PostgreSQL（リレーショナル DB）	Firestore（NoSQL ドキュメント DB）
クエリ言語	SQL（標準的で汎用性が高い）	独自クエリ API
リアルタイム	PostgreSQL の変更検知	ネイティブリアルタイム
認証	GoTrue（メール、OAuth 等）	Firebase Auth（メール、OAuth 等）
ストレージ	S3 互換ストレージ	Cloud Storage
サーバーレス関数	Edge Functions（Deno）	Cloud Functions（Node.js）
料金	無料枠あり、従量課金	無料枠あり、従量課金
オープンソース	はい（セルフホスト可能）	いいえ
ベンダーロックイン	低い（標準 SQL で移行しやすい）	高い（独自仕様が多い）
学習の汎用性	SQL は他の仕事でも使える	Firestore の知識は限定的
複雑なクエリ	JOIN、サブクエリなど強力	限定的（非正規化が必要）
型安全性	CLI で TypeScript 型を自動生成可能	手動で型定義が必要

1.4 PostgreSQL ベースであることの利点

Supabase が PostgreSQL を採用していることには大きなメリットがあります。

1. SQL は業界標準のスキル

SQL（Structured Query Language）は、1970年代から使われ続けているデータベース操作の標準言語です。Web 開発だけでなく、データ分析、機械学習、業務システムなど、あらゆる分野で使われています。Supabase で SQL を学ぶことは、他のどんなプロジェクトでも役立つスキルを身につけることを意味します。

2. リレーショナルデータベースの信頼性

PostgreSQL は 30年以上の歴史を持つ、世界で最も信頼されているオープンソースデータベースの一つです。データの整合性を厳密に保証する仕組み（トランザクション、制約など）が備わっています。

3. 豊富な機能

- **JSON サポート:** NoSQL のような柔軟なデータも格納可能
- **全文検索:** テキスト検索機能を内蔵
- **地理空間データ:** PostGIS 拡張で位置情報も扱える
- **拡張機能:** 数百の拡張機能でほぼあらゆる用途に対応

4. 移行のしやすさ

標準 SQL を使っているため、将来 Supabase 以外のサービス（AWS RDS、Google Cloud SQL、自前サーバーなど）に移行する場合でも、データベースの知識とコードの大部分をそのまま活かします。

1.5 Supabase が提供する機能一覧

機能	説明	本チュートリアルでの使用
Database	PostgreSQL データベース	書籍データの保存
Auth	ユーザー認証（メール、OAuth、マジックリンク等）	第7章で使用予定
Storage	ファイルアップロード・管理	書籍カバー画像の保存（応用編）
Realtime	データ変更のリアルタイム配信	応用編で使用予定
Edge Functions	サーバーレス関数（Deno ランタイム）	応用編で使用予定
Auto-generated API	テーブルから REST / GraphQL API を自動生成	本章でメインで使用
Dashboard	Web ベースの管理画面	テーブル作成・データ確認
CLI	コマンドラインツール	型の自動生成

2. アカウント作成とプロジェクトセットアップ

2.1 Supabase アカウントの作成

ステップ 1: Supabase 公式サイトにアクセス

ブラウザで以下の URL にアクセスします。

```
https://supabase.com
```

トップページが表示されたら、右上の「Start your project」ボタンをクリックします。

ステップ 2: GitHub アカウントでサインアップ

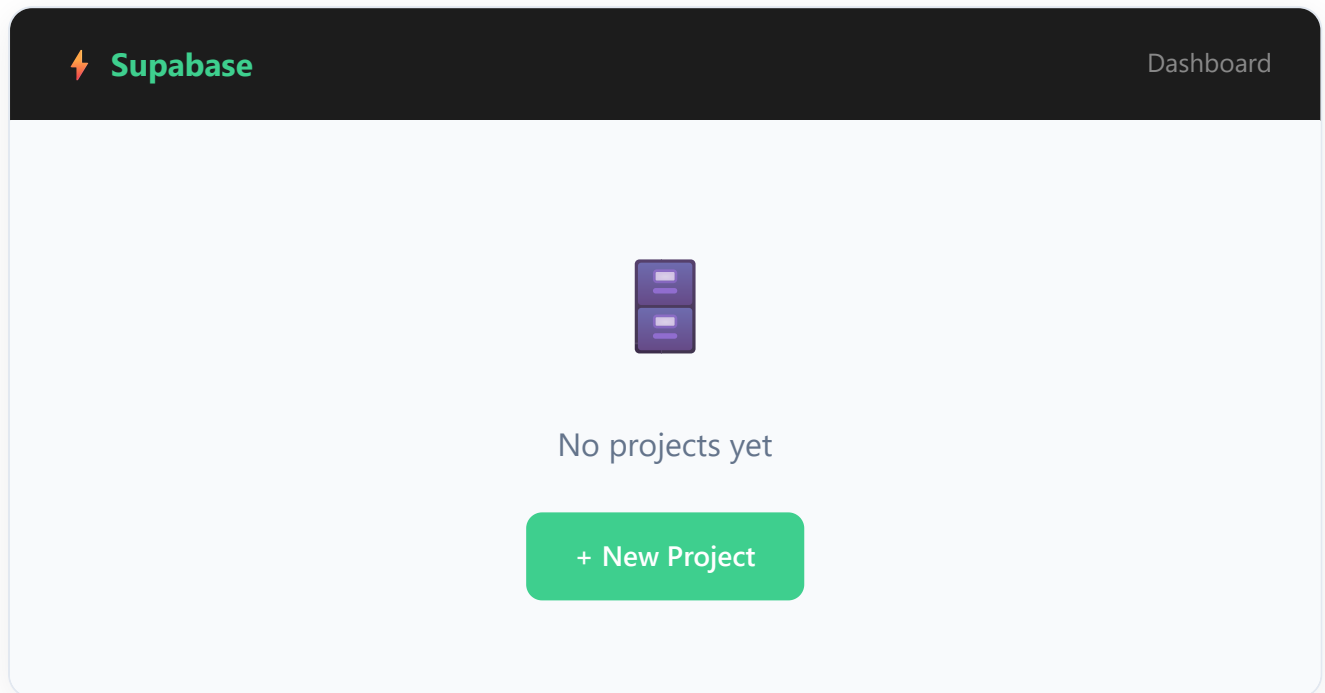
Supabase は GitHub アカウントでのサインアップを推奨しています（GitHub 連携のみ対応）。

1. 「Sign in with GitHub」ボタンをクリック
2. GitHub のログイン画面が表示されるので、自分の GitHub アカウントでログイン
3. Supabase にアクセスを許可するかの確認画面が出るので「Authorize supabase」をクリック
4. Supabase のダッシュボードにリダイレクトできれば成功

GitHub アカウントを持っていない場合: まず <https://github.com> でアカウントを作成してください。GitHub は開発者にとって必須のサービスなので、この機会に作成しておきましょう。

ステップ 3: ダッシュボードの確認

ログイン後、Supabase のダッシュボード画面が表示されます。初回ログイン時はプロジェクトが一つもない状態です。



2.2 新規プロジェクトの作成

ステップ 1: 「New Project」ボタンをクリック

ダッシュボードの「New Project」ボタンをクリックします。

ステップ 2: Organization（組織）の選択

初回の場合は、自動的に個人用の Organization が作られています。そのまま選択してください。

ステップ 3: プロジェクト情報の入力

以下の情報を入力します。

Create a new project

Name

book-manager

Database Password

●●●●●●●●●●●●●●●●

Generate a password

Region

Northeast Asia (Tokyo)

Pricing Plan

Free - \$0/month

Create new project

各項目の詳細:

項目	入力値	説明
Name	book-manager	プロジェクト名。分かりやすい名前をつけましょう
Database Password	(自動生成推奨)	「Generate a password」をクリックして安全なパスワードを生成。 必ずどこかにメモしておくこと
Region	Northeast Asia (Tokyo)	データベースが配置される地域。日本在住なら Tokyo を選択
Pricing Plan	Free	無料プラン。学習には十分な機能が利用可能

ステップ 4: プロジェクトの作成

「Create new project」ボタンをクリックします。プロジェクトのセットアップには 1～2 分程度かかります。画面に進捗バーが表示されるので、完了するまで待ちます。

ステップ 5: プロジェクトダッシュボードの確認

セットアップが完了すると、プロジェクトのダッシュボードが表示されます。



重要: API キーを控えておく

ダッシュボードの「Settings」>「API」から以下の2つの値を確認・メモしてください。後ほどコードから Supabase に接続する際に必要です。

- **Project URL:** `https://xxxxxxxxx.supabase.co` の形式
- **anon (public) key:** `eyJhbGciOiJIUzI1NiIs...` のような長い文字列

2.3 リージョンの選び方

リージョンとは、Supabase のサーバー（データベース）が物理的に配置されるデータセンターの場所です。

選び方の基本原則:

ユーザー（自分やアプリの利用者）に最も近いリージョンを選ぶ

日本在住で、主に日本国内向けのアプリを作る場合は **Northeast Asia (Tokyo)** を選択するのがベストです。

なぜリージョンが重要なのか:

データベースとの通信には物理的な距離に応じた遅延（レイテンシ）が発生します。

リージョン	日本からのおおよその遅延
Tokyo（東京）	5～15ms
Singapore（シンガポール）	60～80ms
US West（米国西部）	100～150ms
US East（米国東部）	150～200ms
EU West（ヨーロッパ西部）	200～300ms

注意: リージョンはプロジェクト作成後に変更できません。間違えた場合は新しいプロジェクトを作り直す必要があります。

2.4 無料プランの制限

Supabase の無料プラン（Free Plan）には以下の制限があります。学習目的であればまったく問題ありません。

項目	無料プランの制限
プロジェクト数	2つまで
データベース容量	500 MB
ストレージ容量	1 GB
帯域幅	5 GB / 月
Edge Function 呼び出し	50万回 / 月
認証ユーザー数	無制限（MAU ベースでない）
一時停止	1週間アクセスがないと自動停止（再起動可）

自動停止について: 無料プランでは、1週間以上ダッシュボードやAPIへのアクセスがないとプロジェクトが自動的に一時停止されます。停止されてもデータは消えません。ダッシュボードにアクセスして「Restore project」ボタンを押せば数分で復旧します。学習中は定期的にアクセスするようにしましょう。

3. データベース設計の基礎

3.1 リレーショナルデータベースとは

リレーショナルデータベース（RDB）とは、データを「テーブル（表）」の形式で管理するデータベースです。Excel のスプレッドシートをイメージすると分かりやすいでしょう。

例: 書籍情報のテーブル

id	title	author	rating
1	ノルウェイの森	村上春樹	5
2	人間失格	太宰治	4
3	1Q84	村上春樹	5

このテーブルの各部分には名前がついています:

- **テーブル（Table）**: データのまとまり全体（上の表そのもの）。「books テーブル」のように名前をつけます
- **行（Row / Record）**: テーブル内の1件のデータ。上の例では3行 = 3冊の書籍データ
- **列（Column / Field）**: データの各項目。上の例では id, title, author, rating の4列

3.2 リレーショナルデータベースの全体像

実際のアプリケーションでは、複数のテーブルが関連し合います。例えば「ユーザー」と「書籍」のように、テーブル間に関係（リレーション）があるのが特徴です。

Relationships: USERS ||--o{ BOOKS (owns) USERS ||--o{ REVIEWS (writes) BOOKS ||--o{ REVIEWS (has) BOOKS }o--|| CATEGORIES (belongs to)

👤 USERS	
🔑 id	uuid (PK)
email	text
name	text
created_at	timestampz

📖 BOOKS	
🔑 id	uuid (PK)
🔗 user_id	uuid (FK)
🔗 category_id	uuid (FK)
title	text
author	text
rating	integer
created_at	timestampz

🏷️ CATEGORIES	
🔑 id	uuid (PK)
name	text

📝 REVIEWS	
🔑 id	uuid (PK)
🔗 user_id	uuid (FK)
🔗 book_id	uuid (FK)
content	text
score	integer
created_at	timestampz

注意: 上の図は一般的な書籍管理システムの例です。本チュートリアルでは、まず **books テーブル1つ** から始め、後の章で認証やリレーションを追加していきます。

3.3 主キー (Primary Key)

主キー (PK: Primary Key) とは、テーブル内の各行を一意に識別するための列です。

なぜ主キーが必要か？

例えば、同じ「村上春樹」の「ノルウェイの森」が2冊登録された場合、どちらを指しているか区別できなくなります。主キーがあれば、`id = 1` と `id = 2` のように一意に特定できます。

主キーのルール:

1. テーブル内でユニーク（重複しない）こと
2. NULL（空）にはできないこと
3. 基本的に変更しないこと

UUID vs 連番（SERIAL）:

方式	例	メリット	デメリット
UUID	550e8400-e29b-41d4-a716-446655440000	衝突の可能性がほぼゼロ。分散システムに適する	長い。人間が読みにくい
SERIAL	1, 2, 3 ...	短い。人間が読みやすい	連番が推測可能。分散環境で衝突しうる

Supabase では UUID がデフォルトで推奨されています。本チュートリアルでも UUID を使用します。

3.4 外部キー（Foreign Key）

外部キー（FK: Foreign Key）とは、別のテーブルの主キーを参照する列です。テーブル間の関連付けに使います。

books テーブル			users テーブル	
id (PK)	user_id(FK)		id (PK)	name
uuid-001	user-AAA	→	user-AAA	田中太郎
uuid-002	user-AAA	→		
uuid-003	user-BBB	→	user-BBB	鈴木花子

上の例では、books テーブルの `user_id` が users テーブルの `id` を参照しています。これにより「どのユーザーがどの本を登録したか」が分かります。

本チュートリアルでは: まだ認証機能を実装していないため、外部キーは後の章で追加します。今回は books テーブル単体で始めます。

3.5 データ型

PostgreSQL で使う主なデータ型を紹介します。

データ型	説明	例
<code>uuid</code>	一意識別子（128ビット）	<code>550e8400-e29b-41d4-a716-446655440000</code>
<code>text</code>	可変長テキスト（長さ制限なし）	<code>'ノルウェイの森'</code>
<code>varchar(n)</code>	可変長テキスト（最大 n 文字）	<code>'abc'</code> （最大n文字）
<code>integer</code>	整数（-2147483648 ～ 2147483647）	<code>42</code>
<code>bigint</code>	大きい整数	<code>9223372036854775807</code>
<code>boolean</code>	真偽値	<code>true</code> / <code>false</code>
<code>date</code>	日付	<code>'2024-01-15'</code>
<code>timestampz</code>	タイムスタンプ（タイムゾーン付き）	<code>'2024-01-15T10:30:00+09:00'</code>
<code>jsonb</code>	JSON データ（バイナリ形式）	<code>'{"key": "value"}'</code>
<code>numeric</code>	精密な小数	<code>3.14</code>

text vs varchar(n) について: PostgreSQL では `text` と `varchar` の性能差はほとんどありません。長さの制限が本当に必要な場合以外は `text` を使うのがシンプルです。Supabase の公式ドキュメントでも `text` が推奨されています。

4. 書籍管理アプリのデータベース設計

4.1 books テーブルの設計

書籍管理アプリに必要なデータを整理し、テーブルを設計しましょう。

books	
🔑 id	uuid (PK, auto)
title NOT NULL	text
author NOT NULL	text
publisher	text
published_date	date
rating CHECK 1-5	integer
status DEFAULT 'want_to_read'	text
notes	text
cover_url	text
created_at DEFAULT NOW()	timestampz
updated_at DEFAULT NOW()	timestampz

4.2 各カラムの設計理由

各カラム（列）がなぜ必要で、なぜそのデータ型を選んだのか、一つずつ解説します。

id - uuid (PK, auto-generated)

```
id uuid DEFAULT gen_random_uuid() PRIMARY KEY
```

- **役割:** 各書籍を一意に識別するための主キー
- **型の理由:** UUID を使うことで、グローバルに一意な ID を自動生成できる。将来的にユーザー認証を追加して複数ユーザーのデータを扱う際にも衝突しない
- **DEFAULT gen_random_uuid():** INSERT 時に自動的に UUID が生成されるため、フロントエンドから ID を指定する必要がない
- **PRIMARY KEY:** この列が主キーであることを示す

title - text (NOT NULL)

```
title text NOT NULL
```

- **役割:** 書籍のタイトル
- **型の理由:** タイトルの長さは予測できないため、可変長の `text` を使用

- **NOT NULL** : 書籍にタイトルがないのはありえないので、必須項目にする。NULL（空）でのINSERTを防ぐ

author - text (NOT NULL)

```
author text NOT NULL
```

- 役割: 著者名
- 型の理由: 著者名も長さが予測できないため **text** を使用
- **NOT NULL** : 著者不明の書籍は「著者不明」と入力する想定。NULL は避ける
- 設計メモ: 本格的なアプリでは著者を別テーブルに分離して多対多のリレーションにしますが、学習用アプリなのでシンプルにテキストで保持します

publisher - text

```
publisher text
```

- 役割: 出版社名
- 型の理由: 出版社名も長さが予測できないため **text**
- NULL 許可: 出版社が分からない、または入力しない場合もあるため、NULL を許可（任意項目）

published_date - date

```
published_date date
```

- 役割: 出版日
- 型の理由: 年月日のみで十分なので **date** 型を使用（時刻は不要）
- NULL 許可: 出版日が不明な場合もあるため、NULL を許可
- 注意: **timestamp** ではなく **date** を使うのは、出版日に時分秒の情報は必要ないため

rating - integer (1-5)

```
rating integer CHECK (rating >= 1 AND rating <= 5)
```

- 役割: 自分の評価（星1〜5つ）
- 型の理由: 1〜5 の整数値なので **integer** が最適
- **CHECK** 制約: データベースレベルで 1〜5 の範囲を強制する。これにより、フロントエンドのバグで不正な値が入ることを防げる
- NULL 許可: まだ読んでいない本は評価できないため、NULL を許可

status - text ('reading' | 'completed' | 'want_to_read')

```
status text DEFAULT 'want_to_read' CHECK (status IN ('reading', 'completed', 'want_to_read'))
```

- 役割: 読書状態の管理
- 型の理由: 決まった値のいずれかなので **text** + **CHECK** 制約で管理
- **DEFAULT 'want_to_read'**: 新しく登録した本はデフォルトで「読みたい」状態にする
- **CHECK 制約**: 許可された3つの値以外は登録できないようにする
- 各値の意味:
 - **reading**: 現在読んでいる
 - **completed**: 読了済み
 - **want_to_read**: 読みたい

なぜ **enum** 型ではなく **text** + **CHECK** か? PostgreSQL には **enum** 型がありますが、後から値を削除するのが難しいという問題があります。**text** + **CHECK** であれば、将来的に **CHECK** 制約を変更するだけで値を追加・変更・削除できます。

notes - text

```
notes text
```

- 役割: 読書メモ・感想
- 型の理由: 長文を格納できる **text** を使用
- NULL 許可: メモは任意項目

cover_url - text

```
cover_url text
```

- 役割: 書籍の表紙画像の URL
- 型の理由: URL は文字列なので **text**
- NULL 許可: 画像がない場合もある
- 設計メモ: 将来的に Supabase Storage を使って画像をアップロードする機能を追加する際、ここに Storage の URL を格納する

created_at - timestampz

```
created_at timestampz DEFAULT NOW()
```

- **役割:** データが作成された日時
- **型の理由:** タイムゾーン情報を含む `timestampz` (timezone 付き timestamp) を使用。世界中どこからアクセスしても正しい時刻が記録される
- `DEFAULT NOW()` : INSERT 時に自動的に現在時刻が設定される

`updated_at` - `timestampz`

```
updated_at timestampz DEFAULT NOW()
```

- **役割:** データが最後に更新された日時
- **型の理由:** `created_at` と同じく `timestampz`
- `DEFAULT NOW()` : INSERT 時に自動的に現在時刻が設定される
- **注意:** UPDATE 時に自動更新するにはトリガーが必要 (後述)

4.3 SQL でのテーブル作成

以下の SQL で books テーブルを作成します。Supabase の SQL Editor で実行してください。

ステップ 1: SQL Editor を開く

Supabase ダッシュボードの左メニューから「SQL Editor」をクリックします。

ステップ 2: 新しいクエリを作成

「New query」ボタンをクリックし、以下の SQL をコピー & ペーストします。

```

-- =====
-- books テーブルの作成 - 書籍管理アプリのメインテーブル
-- =====

-- SQL の構文: CREATE TABLE テーブル名 ( カラム定義1, カラム定義2, ... );
-- カラム定義: 「カラム名 データ型 制約」の3つを並べて書く。
-- 行末のセミコロン ; が SQL文の終わり。
-- -- (ハイフン2つ) で始まる行はコメント (実行時に無視される)。
-- =====

CREATE TABLE books (

-- (1) 主キー(PRIMARY KEY): レコードを一意に識別する列。
--     uuid                : 文字列の一種で、世界中で重複しないIDを表す型
--     gen_random_uuid()   : Postgres組み込み関数。新しいUUIDを1つ生成する
--     DEFAULT ...         : INSERT時にこの値が省略されたら自動で入る初期値
--     PRIMARY KEY         : 主キー宣言。NULL不可・重複不可になる
id uuid DEFAULT gen_random_uuid() PRIMARY KEY,

-- (2) 書籍情報 (必須)
--     text                : 可変長の文字列 (長さ制限なし)。MySQL の VARCHAR に相当
--     NOT NULL            : 「NULL (値なし) を許可しない」制約
title text NOT NULL,
author text NOT NULL,

-- (3) 書籍情報 (任意)
--     NOT NULL を付けないので、INSERT時に省略可能 → NULLが入る
publisher text,
--     date                : 「年月日」だけを保存する型。時刻は持たない
published_date date,

-- (4) 読書管理
--     integer             : 整数型
--     CHECK ( ... )      : 値が条件を満たさなければINSERT/UPDATEを拒否する制約
--     ここでは「rating は 1以上かつ5以下」を強制する
rating integer CHECK (rating >= 1 AND rating <= 5),

--     DEFAULT 'want_to_read' : INSERT時に省略されたら 'want_to_read' を入れる
--     status IN ('reading', 'completed', 'want_to_read')
--     → これらの値以外は登録不可 (typoや不正値を防ぐ)
status text DEFAULT 'want_to_read'
    CHECK (status IN ('reading', 'completed', 'want_to_read')),

notes text,

-- (5) 画像URL
cover_url text,

-- (6) タイムスタンプ
--     timestampz          : 「タイムゾーン付きの日時」を表す型 (推奨)

```

```

--      NOW()      : 現在の日時を返す関数
--      DEFAULT NOW() で「INSERT時に自動で現在時刻が入る」
created_at timestampz DEFAULT NOW(),
updated_at timestampz DEFAULT NOW()
);

-- ▼ 実行結果 (成功時、Supabase SQL Editor の出力)
--      Success. No rows returned
--      → CREATE 系の文は「行を返さない」ので「成功・行なし」と表示される。

-- =====
--      updated_at を自動更新するトリガー
--      =====
--      「レコードが UPDATE されるたびに updated_at を現在時刻にする」仕組みを
--      DBレベルで実現する。アプリ側でいちいち書かなくていい。
--      =====

-- (1) まず「呼ばれた時に動く関数」を定義する。
--      CREATE OR REPLACE FUNCTION:
--          同名の関数が既にあれば置き換える、無ければ新規作成。
--      RETURNS TRIGGER:
--          戻り値の型がトリガー専用の特殊な値であることを示す。
--      $$ ... $$:
--          関数本体を囲む区切り。シングルクォートを多用するときに便利。
--      LANGUAGE plpgsql:
--          関数本体は PL/pgSQL (Postgres の手続き型言語) で書く宣言。
CREATE OR REPLACE FUNCTION update_updated_at_column()
RETURNS TRIGGER AS $$
BEGIN
    -- NEW は「これから書き込まれる新しい行」の擬似変数
    -- そのレコードの updated_at を現在時刻に書き換える
    NEW.updated_at = NOW();
    -- 書き換えた NEW を返すことでDBに反映される
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- (2) 上の関数を「books テーブルが UPDATE される直前」に呼ぶよう紐付ける
--      BEFORE UPDATE: 「UPDATE実行の直前に呼ぶ」 タイミング指定
--      FOR EACH ROW : 「行ごとに1回呼ぶ」 (複数行同時更新時も全行に効く)
--      EXECUTE FUNCTION ...: 実行する関数の指定
CREATE TRIGGER update_books_updated_at
    BEFORE UPDATE ON books
    FOR EACH ROW
    EXECUTE FUNCTION update_updated_at_column();

-- ▼ これ以降、UPDATE文を実行するたびに自動で updated_at が NOW() に書き換わる。
--      例: UPDATE books SET title='新タイトル' WHERE id='...';
--      → 自分で updated_at を指定しなくても、自動的に現在時刻が入る。

```

```
-- =====
-- テーブルとカラムにコメントを追加
-- Dashboard での表示が分かりやすくなる
-- =====

COMMENT ON TABLE books IS '書籍管理テーブル';
COMMENT ON COLUMN books.id IS '書籍ID（自動生成）';
COMMENT ON COLUMN books.title IS '書籍タイトル';
COMMENT ON COLUMN books.author IS '著者名';
COMMENT ON COLUMN books.publisher IS '出版社';
COMMENT ON COLUMN books.published_date IS '出版日';
COMMENT ON COLUMN books.rating IS '評価（1-5）';
COMMENT ON COLUMN books.status IS '読書状態（reading/completed/want_to_read）';
COMMENT ON COLUMN books.notes IS 'メモ・感想';
COMMENT ON COLUMN books.cover_url IS '表紙画像URL';
COMMENT ON COLUMN books.created_at IS '作成日時';
COMMENT ON COLUMN books.updated_at IS '更新日時';
```

ステップ 3: SQL を実行

SQL Editor の右下にある「Run」ボタン（または **Ctrl + Enter** / **Cmd + Enter**）をクリックして実行します。

```
Success. No rows returned.
```

と表示されれば成功です。

ステップ 4: テーブルの確認

左メニューの「Table Editor」をクリックすると、作成した **books** テーブルが表示されているはずです。テーブル名をクリックすると、カラムの一覧やデータ（まだ空）が確認できます。

4.4 Supabase Dashboard での GUI 操作手順（代替手段）

SQL を使わずに、Dashboard の GUI でテーブルを作成することもできます。ここではその手順を説明します。

推奨: SQL での作成を推奨しますが、GUI での操作も知っておくと便利です。

ステップ 1: Table Editor を開く

左メニューの「Table Editor」をクリックします。

ステップ 2: 新しいテーブルを作成

「Create a new table」ボタンをクリックします。

ステップ 3: テーブル名の入力

- Name: **books** と入力

- **Description:** 書籍管理テーブル と入力（任意）
- **Enable Row Level Security (RLS):** チェックを入れる（後で設定）

ステップ 4: カラムの追加

デフォルトで **id** (uuid, Primary Key) と **created_at** (timestampz) が用意されています。残りのカラムを以下の手順で追加します:

1. 「Add column」ボタンをクリック
2. 以下の情報を入力:

```
Column Name: title
Type: text
Default Value: (空欄)
Primary: [ ]
Nullable: [ ] ← チェックを外す (NOT NULL にする)
Unique: [ ]
[Save]
```

3. 同様に、残りのカラムも一つずつ追加: - **author** (text, NOT NULL) - **publisher** (text, Nullable) - **published_date** (date, Nullable) - **rating** (int4, Nullable) - **status** (text, Default: 'want_to_read') - **notes** (text, Nullable) - **cover_url** (text, Nullable) - **updated_at** (timestampz, Default: now())
4. すべてのカラムを追加したら「Save」ボタンをクリック

注意: GUI では CHECK 制約やトリガーを設定できないため、別途 SQL Editor で以下を実行する必要があります:


```

-- CHECK 制約の追加
ALTER TABLE books
  ADD CONSTRAINT books_rating_check CHECK (rating >= 1 AND rating <= 5);

ALTER TABLE books
  ADD CONSTRAINT books_status_check CHECK (status IN ('reading', 'completed', 'want_to_read'));

-- updated_at 自動更新トリガーの追加
CREATE OR REPLACE FUNCTION update_updated_at_column()
RETURNS TRIGGER AS $$
BEGIN
  NEW.updated_at = NOW();
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER update_books_updated_at
BEFORE UPDATE ON books
FOR EACH ROW
EXECUTE FUNCTION update_updated_at_column();

```

5. Row Level Security (RLS)

5.1 RLS とは

Row Level Security (RLS: 行レベルセキュリティ) とは、PostgreSQL の機能で、テーブルの各行に対して「誰がどの行にアクセスできるか」を制御する仕組みです。

なぜ RLS が必要か？

Supabase はフロントエンド（ブラウザ）から直接データベースにアクセスします。つまり、API キー（anon key）はブラウザの JavaScript コードに埋め込まれ、誰でも見ることができます。

通常のサーバー構成:

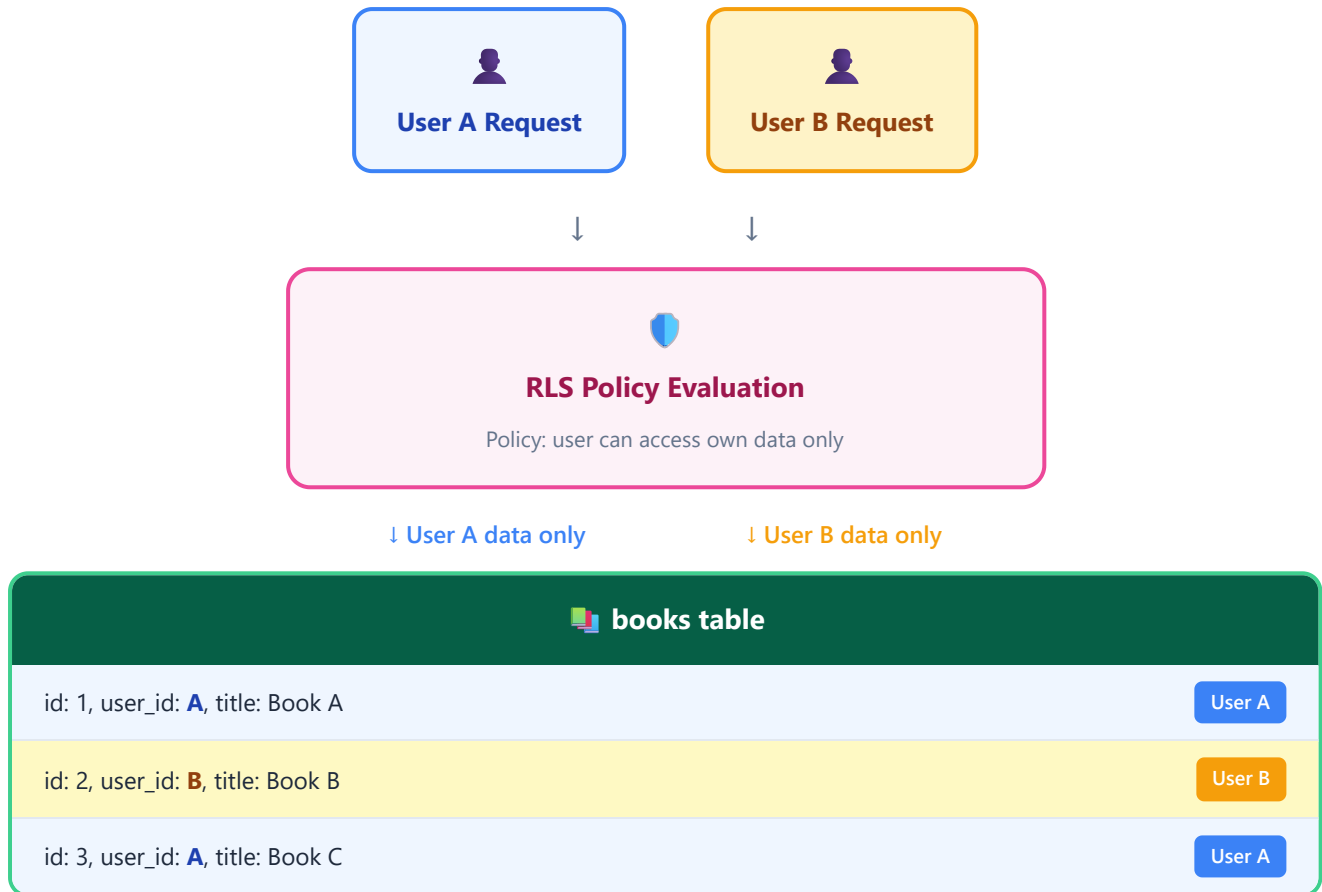
ブラウザ → API サーバー（ここでアクセス制御）→ データベース

Supabase の構成:

ブラウザ → Supabase API（ここに RLS でアクセス制御）→ データベース

RLS がなければ、anon key を知っている人は誰でもすべてのデータを読み書きできてしまいます。RLS を有効にすることで、「どのユーザーがどのデータにアクセスできるか」をデータベースレベルで制御できます。

5.2 RLS の仕組み（図解）



5.3 書籍管理アプリでの RLS 設定（簡易版）

本チュートリアルではまだ認証機能を実装していないため、まずは **誰でも全データにアクセスできる** シンプルなポリシーを設定します。認証機能の実装後に、より厳密なポリシーに変更します。

SQL Editor で以下を実行してください:

```

-- =====
-- RLS (Row Level Security) の設定
-- =====

-- 「テーブルの行ごとに、誰がアクセスできるかを定める」セキュリティ機能。
-- Supabase ではデフォルトでRLSが無効。
-- ALTER TABLE で有効化し、CREATE POLICY で「許可ルール」を1つずつ追加していく。
-- =====

-- (1) books テーブルの RLS を有効化する
--     ALTER TABLE          : 既存テーブルの設定を変更するSQL文
--     ENABLE ROW LEVEL SECURITY : RLSを ON にする
--     ※ RLSを有効にしたあと、ポリシーを1つも作っていないと「全部拒否」になる
ALTER TABLE books ENABLE ROW LEVEL SECURITY;

-- (2) SELECT (読み取り) を全員に許可するポリシー
--     CREATE POLICY "ポリシー名" ON テーブル名
--     FOR 操作種別 ← どの操作 (SELECT/INSERT/UPDATE/DELETE/ALL) に効くか
--     USING (条件) ← どの「既存の行」がこの操作に使えるかを定めるブール式
--     true は「常に真」 → 全行に対して許可するという意味
CREATE POLICY "Allow public read access"
ON books
FOR SELECT
USING (true);

-- (3) INSERT (新規作成) を全員に許可するポリシー
--     INSERTでは「これから書き込まれる新行」が対象なので USING ではなく
--     WITH CHECK を使う。
--     WITH CHECK (true) → どんな値の行でも書き込みOK
CREATE POLICY "Allow public insert access"
ON books
FOR INSERT
WITH CHECK (true);

-- (4) UPDATE (更新) を全員に許可するポリシー
--     UPDATEは「既存の行を選んで」「新しい値で上書き」の2フェーズなので
--     USING (読み取り対象の判定) と WITH CHECK (書き込み内容の判定) の両方を書く。
CREATE POLICY "Allow public update access"
ON books
FOR UPDATE
USING (true)          -- 全ての既存行を更新対象にできる
WITH CHECK (true);   -- どんな新しい値でも書き込みOK

-- (5) DELETE (削除) を全員に許可するポリシー
--     DELETEは新規行を作らないので WITH CHECK は不要、USINGだけで判定する。
CREATE POLICY "Allow public delete access"

```

```
ON books
FOR DELETE
USING (true);
```

```
-- ▼ 実行後の状態
-- このテーブルは「誰でもCRUD可能な開発用設定」になる。
-- 本番運用や認証実装後は、この4つを DROP POLICY で削除し、
-- auth.uid() = user_id のような「自分のレコードだけ」ポリシーに差し替える。
```

各ポリシーの解説:

ポリシー名	対象操作	USING	WITH CHECK	意味
Allow public read access	SELECT	true	-	すべての行を読み取り可能
Allow public insert access	INSERT	-	true	すべての行を挿入可能
Allow public update access	UPDATE	true	true	すべての行を更新可能
Allow public delete access	DELETE	true	-	すべての行を削除可能

USING と **WITH CHECK** の違い:

- **USING**: 既存の行に対する条件 (SELECT, UPDATE, DELETE で使用)。「どの行が見えるか」
- **WITH CHECK**: 新しく書き込まれる行に対する条件 (INSERT, UPDATE で使用)。「どの行を書き込めるか」

5.4 認証実装後の RLS ポリシー (参考)

第7章で認証機能を追加した後は、以下のようなポリシーに変更します。参考として掲載しておきます。

```

-- 認証済みユーザーが自分のデータのみアクセスできるポリシー
-- （第7章で実装予定。今は実行しないでください）

-- 既存のポリシーを削除
-- DROP POLICY "Allow public read access" ON books;
-- DROP POLICY "Allow public insert access" ON books;
-- DROP POLICY "Allow public update access" ON books;
-- DROP POLICY "Allow public delete access" ON books;

-- ユーザーは自分のデータのみ SELECT 可能
-- CREATE POLICY "Users can view own books"
--   ON books
--   FOR SELECT
--   USING (auth.uid() = user_id);

-- ユーザーは自分の user_id でのみ INSERT 可能
-- CREATE POLICY "Users can insert own books"
--   ON books
--   FOR INSERT
--   WITH CHECK (auth.uid() = user_id);

-- ユーザーは自分のデータのみ UPDATE 可能
-- CREATE POLICY "Users can update own books"
--   ON books
--   FOR UPDATE
--   USING (auth.uid() = user_id)
--   WITH CHECK (auth.uid() = user_id);

-- ユーザーは自分のデータのみ DELETE 可能
-- CREATE POLICY "Users can delete own books"
--   ON books
--   FOR DELETE
--   USING (auth.uid() = user_id);

```

6. Supabase Client のセットアップ

6.1 @supabase/supabase-js のインストール

プロジェクトのルートディレクトリで以下のコマンドを実行します。

```
npm install @supabase/supabase-js
```

正常にインストールされると、`package.json` の `dependencies` に追加されます。

```
{
  "dependencies": {
    "@supabase/supabase-js": "^2.x.x"
  }
}
```

6.2 環境変数の設定

Supabase に接続するために必要な情報を環境変数として設定します。環境変数を使うことで、API キーなどの秘密情報をコードにハードコードせずに管理できます。

ステップ 1: Supabase Dashboard から情報を取得

1. Supabase Dashboard にログイン
2. 対象プロジェクトを選択
3. 左メニューの「Settings」(歯車アイコン) をクリック
4. 「API」をクリック
5. 以下の2つの値をコピー: - **Project URL:** `https://xxxxxxxx.supabase.co` - **anon public key:** `eyJhbGciOiJIUzI1NiIs...` (長い文字列)

ステップ 2: `.env.local` ファイルの作成

プロジェクトのルートディレクトリに `.env.local` ファイルを作成します。

```
# .env.local  
# Supabase の接続情報  
  
NEXT_PUBLIC_SUPABASE_URL=https://xxxxxxx.supabase.co  
NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9. xxxxxxxxxxxxxxxxxxxxx
```

A horizontal scrollbar at the bottom of the terminal window, indicating that the output has been scrolled horizontally.

重要な注意点:

1. `xxxxxxx` の部分は、あなたのプロジェクトの実際の値に置き換えてください
2. `NEXT_PUBLIC_` プレフィックスは Next.js で環境変数をブラウザ側で使うために必要です
3. `.env.local` は 絶対に Git にコミットしないでください。 `.gitignore` に含まれていることを確認しましょう

1. `xxxxxxx` の部分は、あなたのプロジェクトの実際の値に置き換えてください
2. `NEXT_PUBLIC_` プレフィックスは Next.js で環境変数をブラウザ側で使うために必要です
3. `.env.local` は **絶対に Git にコミットしないでください**。`.gitignore` に含まれていることを確認しましょう

ステップ 3: **.gitignore** に追加されていることを確認

Next.js のプロジェクトを `create-next-app` で作成した場合、`.env.local` はデフォルトで `.gitignore` に含まれています。念のため確認しましょう。

```
# .gitignore に以下が含まれていることを確認
.env*.local
```

もし含まれていなければ、`.gitignore` に以下を追加してください:

```
# 環境変数ファイル
.env*.local
```

6.3 Supabase クライアントの初期化

Supabase に接続するためのクライアントを作成します。

ステップ 1: ファイルの作成

`src/lib/supabase.ts` ファイルを作成します。

```
mkdir -p src/lib
```

ステップ 2: クライアントコードの記述

```

// =====
// ファイルパス: src/lib/supabase.ts
// 役割      : アプリ全体で使う「Supabaseクライアント」を1個だけ作って共有する
// -----
// このファイルを作っておくと、他のファイルから
//   import { supabase } from "@lib/supabase";
// と書くだけでDB操作できるようになる。
// =====

// (1) Supabase クライアント作成関数を取り込む
//   @supabase/supabase-js は Supabase の公式SDK (npm/パッケージ)
import { createClient } from '@supabase/supabase-js';

// (2) DBスキーマから自動生成した型を取り込む
//   `import type` は「型情報だけ取り込み、実行時のJSには残さない」記法
//   @ は src/ を表すパスエイリアス (tsconfig.json の paths で設定済み)
import type { Database } from '@types/supabase';

// (3) 環境変数から接続情報を取得
//   process.env はビルド時に Next.js が値を埋め込む
//   NEXT_PUBLIC_ プレフィックスはブラウザ側にも値を渡すための約束（ない場合は
//   サーバーでしか参照できない）
const supabaseUrl = process.env.NEXT_PUBLIC_SUPABASE_URL;
const supabaseAnonKey = process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY;

// (4) 環境変数が未設定の場合は早期にエラーを出す (Fail-Fast)
//   これを書かないと「なぜか動かない」と原因不明になりやすい。
//   エラー文に「.env.local を確認」と書いて初心者にはやさしくする。
if (!supabaseUrl) {
  throw new Error(
    'NEXT_PUBLIC_SUPABASE_URL が設定されていません。 .env.local ファイルを確認してください。'
  );
}

if (!supabaseAnonKey) {
  throw new Error(
    'NEXT_PUBLIC_SUPABASE_ANON_KEY が設定されていません。 .env.local ファイルを確認してください。'
  );
}

// (5) Supabase クライアントを作って export する
//   <Database> はジェネリクス型引数。これを渡しておく
//   supabase.from('books').select('*') と書いたときに
//   VS Code が books の存在やカラム名を補完・検証してくれる。
//   export const にすることで、他のファイルから
//   import { supabase } from "@lib/supabase";
//   で使えるようになる。
export const supabase = createClient<Database>(supabaseUrl, supabaseAnonKey);

```



```
// ▼ 使用例 (別ファイルから)
// import { supabase } from "@lib/supabase";
// const { data, error } = await supabase.from("books").select("*");
// console.log(data); // Book[] 型 (自動推論)
```

コードの解説:

1. `createClient`: Supabase クライアントを作成する関数。 `@supabase/supabase-js` からインポート
2. `Database`: TypeScript の型定義 (次のセクションで生成)。これにより、テーブル名やカラム名の補完が効くようになる
3. `process.env.NEXT_PUBLIC_SUPABASE_URL`: 環境変数から Supabase の URL を取得
4. `process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY`: 環境変数から API キーを取得
5. エラーチェック: 環境変数が未設定の場合、分かりやすいエラーメッセージを表示

6.4 TypeScript 型の自動生成 (supabase gen types)

Supabase CLI を使うと、データベースのスキーマ (テーブル定義) から TypeScript の型を自動生成できます。これにより、コード内でテーブル名やカラム名の入力補完が効くようになり、タイプミスを防げます。

ステップ 1: Supabase CLI のインストール

```
npm install -D supabase
```

ステップ 2: Supabase CLI にログイン

```
npx supabase login
```

ブラウザが開き、Supabase へのログインが求められます。認証を完了してください。

ステップ 3: 型定義ファイルの生成

```
# プロジェクトのリファレンス ID を確認
# Supabase Dashboard の URL: https://supabase.com/dashboard/project/[ここがリファレンスID]
# または Settings > General > Reference ID で確認

npx supabase gen types typescript --project-id "あなたのプロジェクトID" --schema public > src
```

注意: あなたのプロジェクトID は Supabase Dashboard の URL に含まれる英数字の文字列です。

ステップ 4: 型定義ディレクトリの作成

事前にディレクトリを作成しておく必要があります:

```
mkdir -p src/types
```

ステップ 5: 生成される型定義ファイルの確認

自動生成された `src/types/supabase.ts` は以下のような内容になります（一部抜粋）：

```
// src/types/supabase.ts
// このファイルは自動生成されます。手動で編集しないでください。
```

```
export type Json =
  | string
  | number
  | boolean
  | null
  | { [key: string]: Json | undefined }
  | Json[];

export type Database = {
  public: {
    Tables: {
      books: {
        Row: {
          id: string;
          title: string;
          author: string;
          publisher: string | null;
          published_date: string | null;
          rating: number | null;
          status: string | null;
          notes: string | null;
          cover_url: string | null;
          created_at: string | null;
          updated_at: string | null;
        };
        Insert: {
          id?: string;
          title: string;
          author: string;
          publisher?: string | null;
          published_date?: string | null;
          rating?: number | null;
          status?: string | null;
          notes?: string | null;
          cover_url?: string | null;
          created_at?: string | null;
          updated_at?: string | null;
        };
        Update: {
          id?: string;
          title?: string;
          author?: string;
          publisher?: string | null;
          published_date?: string | null;
          rating?: number | null;
          status?: string | null;
        };
      };
    };
  };
};
```

```

    notes?: string | null;
    cover_url?: string | null;
    created_at?: string | null;
    updated_at?: string | null;
  };
};
};
Views: {
  [_ in never]: never;
};
Functions: {
  [_ in never]: never;
};
Enums: {
  [_ in never]: never;
};
};
};
};

```

型定義の3つの種類:

型	説明	使い方
Row	SELECT で取得される行の型。全カラムが含まれる	データ表示時に使用
Insert	INSERT で送信するデータの型。デフォルト値があるカラムは optional (?)	データ作成時に使用
Update	UPDATE で送信するデータの型。全カラムが optional	データ更新時に使用

ステップ 6: 便利な型エイリアスの作成

コードで使いやすいように、型のエイリアス（別名）を作成しておきましょう。

```
// src/types/book.ts

import type { Database } from './supabase';

// books テーブルの型エイリアス
export type Book = Database['public']['Tables']['books']['Row'];
export type BookInsert = Database['public']['Tables']['books']['Insert'];
export type BookUpdate = Database['public']['Tables']['books']['Update'];

// 読書状態の型
export type BookStatus = 'reading' | 'completed' | 'want_to_read';

// 読書状態の日本語ラベル
export const BOOK_STATUS_LABELS: Record<BookStatus, string> = {
  reading: '読書中',
  completed: '読了',
  want_to_read: '読みたい',
};
```

ステップ 7: package.json にスクリプトを追加

型生成を簡単に実行できるよう、`package.json` にスクリプトを追加します。

```
{
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start",
    "lint": "next lint",
    "gen:types": "supabase gen types typescript --project-id \"あなたのプロジェクトID\" --schema public"
  }
}
```

これにより、テーブル定義を変更した後は以下のコマンドで型を再生成できます:

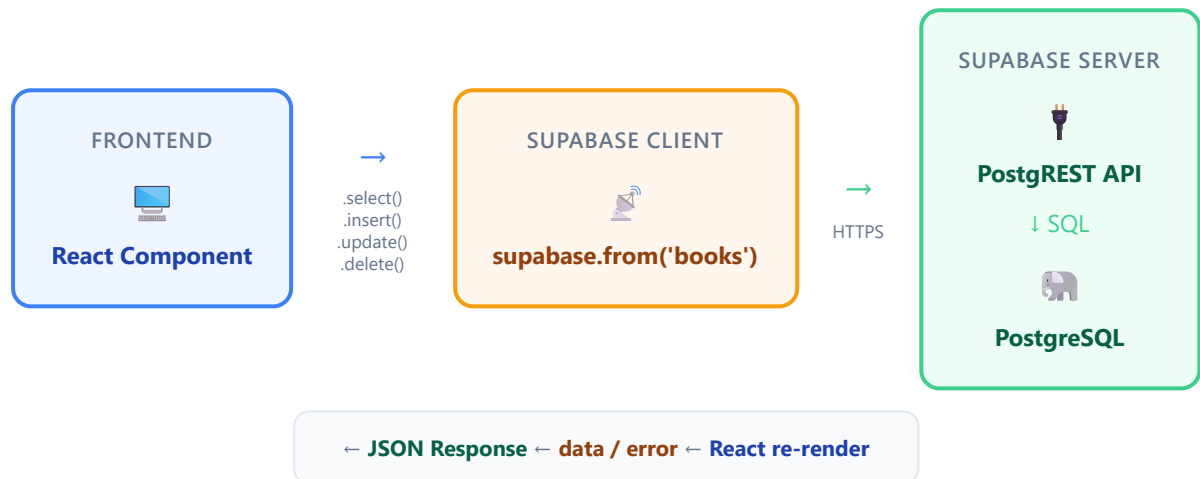
```
npm run gen:types
```

7. Supabase の基本操作（CRUD）

CRUD とは、データベースの4つの基本操作の頭文字をとったものです。

操作	意味	SQL	Supabase メソッド
Create	作成	INSERT	<code>.insert()</code>
Read	読み取り	SELECT	<code>.select()</code>
Update	更新	UPDATE	<code>.update()</code>
Delete	削除	DELETE	<code>.delete()</code>

以下のデータフローを意識しながら、各操作を学んでいきましょう。



7.1 SELECT（データの読み取り）

全件取得

```
// すべての書籍を取得
const { data, error } = await supabase
  .from('books')
  .select('*');

if (error) {
  console.error('エラー:', error.message);
  return;
}

console.log('書籍一覧:', data);
// 結果例:
// [
//   {
//     id: '550e8400-e29b-41d4-a716-446655440000',
//     title: 'ノルウェイの森',
//     author: '村上春樹',
//     publisher: '講談社',
//     published_date: '1987-09-04',
//     rating: 5,
//     status: 'completed',
//     notes: '名作。何度も読み返したい。',
//     cover_url: null,
//     created_at: '2024-01-15T10:30:00.000Z',
//     updated_at: '2024-01-15T10:30:00.000Z'
//   },
//   ...
// ]
```

特定のカラムのみ取得

```
// タイトルと著者のみ取得（データ転送量を減らせる）
const { data, error } = await supabase
  .from('books')
  .select('title, author, rating');

// 結果例:
// [
//   { title: 'ノルウェイの森', author: '村上春樹', rating: 5 },
//   { title: '人間失格', author: '太宰治', rating: 4 },
//   ...
// ]
```

条件付き取得（フィルタリング）

```
// 評価が4以上の書籍のみ取得
const { data, error } = await supabase
  .from('books')
  .select('*')
  .gte('rating', 4); // gte = greater than or equal (以上)

// 結果例:
// [
//   { title: 'ノルウェイの森', rating: 5, ... },
//   { title: '人間失格', rating: 4, ... },
// ]
```

```
// 読書中の書籍のみ取得
const { data, error } = await supabase
  .from('books')
  .select('*')
  .eq('status', 'reading'); // eq = equal (等しい)
```

```
// タイトルに「村上」を含む書籍を検索
const { data, error } = await supabase
  .from('books')
  .select('*')
  .ilike('author', '%村上%'); // ilike = 大文字小文字を区別しない部分一致
```

主なフィルタメソッド一覧:

メソッド	SQL 相当	説明	例
<code>.eq(column, value)</code>	<code>= value</code>	等しい	<code>.eq('status', 'reading')</code>
<code>.neq(column, value)</code>	<code>!= value</code>	等しくない	<code>.neq('status', 'completed')</code>
<code>.gt(column, value)</code>	<code>> value</code>	より大きい	<code>.gt('rating', 3)</code>
<code>.gte(column, value)</code>	<code>>= value</code>	以上	<code>.gte('rating', 4)</code>
<code>.lt(column, value)</code>	<code>< value</code>	より小さい	<code>.lt('rating', 3)</code>
<code>.lte(column, value)</code>	<code><= value</code>	以下	<code>.lte('rating', 2)</code>
<code>.like(column, pattern)</code>	<code>LIKE pattern</code>	パターン一致	<code>.like('title', '%森%')</code>
<code>.ilike(column, pattern)</code>	<code>ILIKE pattern</code>	パターン一致（大文字小文字無視）	<code>.ilike('author', '%murakami%')</code>
<code>.is(column, value)</code>	<code>IS value</code>	NULL チェック	<code>.is('notes', null)</code>
<code>.in(column, values)</code>	<code>IN (values)</code>	複数値のいずれか	<code>.in('status', ['reading', 'completed'])</code>

ソート

```
// 評価が高い順にソート
const { data, error } = await supabase
  .from('books')
  .select('*')
  .order('rating', { ascending: false }); // ascending: false = 降順

// 結果例:
// [
//   { title: 'ノルウェイの森', rating: 5, ... },
//   { title: '人間失格', rating: 4, ... },
//   { title: 'こころ', rating: 3, ... },
// ]
```

```
// 作成日時の新しい順にソート
const { data, error } = await supabase
  .from('books')
  .select('*')
  .order('created_at', { ascending: false });
```

ページネーション

```
// 1ページあたり10件、1ページ目を取得
const pageSize = 10;
const page = 1;

const { data, error, count } = await supabase
  .from('books')
  .select('*', { count: 'exact' }) // count: 'exact' で総件数も取得
  .order('created_at', { ascending: false })
  .range((page - 1) * pageSize, page * pageSize - 1); // range(開始, 終了)

console.log('データ:', data); // 10件のデータ
console.log('総件数:', count); // テーブル内の全件数
console.log('総ページ数:', Math.ceil((count ?? 0) / pageSize));
```

1件だけ取得

```
// ID を指定して1件取得
const { data, error } = await supabase
  .from('books')
  .select('*')
  .eq('id', '550e8400-e29b-41d4-a716-446655440000')
  .single(); // single() で1件のみ取得（配列ではなくオブジェクトが返る）

// 結果例:
// {
//   id: '550e8400-e29b-41d4-a716-446655440000',
//   title: 'ノルウェイの森',
//   ...
// }
```

.single() の注意点: **.single()** は結果が0件または2件以上の場合にエラーを返します。必ず1件だけ返ることが保証される場面（主キーで検索する場合など）でのみ使用してください。0件の可能性がある場合は **.maybeSingle()** を使用します。

7.2 INSERT（データの作成）

単一行の挿入

```
import type { BookInsert } from '@types/book';

// 新しい書籍を1件追加
const newBook: BookInsert = {
  title: 'ノルウェイの森',
  author: '村上春樹',
  publisher: '講談社',
  published_date: '1987-09-04',
  rating: 5,
  status: 'completed',
  notes: '名作。何度も読み返したい。',
};

const { data, error } = await supabase
  .from('books')
  .insert(newBook)
  .select(); // .select() を付けると、挿入したデータが返る

if (error) {
  console.error('挿入エラー:', error.message);
  return;
}

console.log('挿入されたデータ:', data);
// 結果例:
// [
//   {
//     id: '自動生成されたUUID',
//     title: 'ノルウェイの森',
//     author: '村上春樹',
//     publisher: '講談社',
//     published_date: '1987-09-04',
//     rating: 5,
//     status: 'completed',
//     notes: '名作。何度も読み返したい。',
//     cover_url: null,
//     created_at: '2024-01-15T10:30:00.000Z',
//     updated_at: '2024-01-15T10:30:00.000Z'
//   }
// ]
```

.select() を付ける理由: **.insert()** だけだとレスポンスにデータが含まれません。挿入したデータ（自動生成された id や created_at を含む）を取得したい場合は **.select()** を付けてください。

複数行の挿入

```
// 複数の書籍を一度に追加
const newBooks: BookInsert[] = [
  {
    title: '人間失格',
    author: '太宰治',
    publisher: '新潮社',
    published_date: '1948-06-01',
    rating: 4,
    status: 'completed',
  },
  {
    title: 'こころ',
    author: '夏目漱石',
    publisher: '岩波書店',
    published_date: '1914-09-01',
    rating: 5,
    status: 'reading',
  },
  {
    title: '銀河鉄道の夜',
    author: '宮沢賢治',
    status: 'want_to_read', // 最低限 title と author があれば OK
  },
];

const { data, error } = await supabase
  .from('books')
  .insert(newBooks)
  .select();

if (error) {
  console.error('挿入エラー:', error.message);
  return;
}

console.log(`${data.length}件の書籍を追加しました`);
```

7.3 UPDATE (データの更新)

```
import type { BookUpdate } from '@types/book';

// 特定の書籍の情報を更新
const bookId = '550e8400-e29b-41d4-a716-446655440000';

const updates: BookUpdate = {
  rating: 4,
  status: 'completed',
  notes: '読了。面白かった！',
};

const { data, error } = await supabase
  .from('books')
  .update(updates)
  .eq('id', bookId) // 必ず条件を指定すること！
  .select();

if (error) {
  console.error('更新エラー:', error.message);
  return;
}

console.log('更新されたデータ:', data);
// 結果例:
// [
//   {
//     id: '550e8400-e29b-41d4-a716-446655440000',
//     title: 'ノルウェイの森',
//     rating: 4,           ← 更新された
//     status: 'completed', ← 更新された
//     notes: '読了。面白かった！', ← 更新された
//     updated_at: '2024-01-16T15:00:00.000Z', ← トリガーにより自動更新
//     ...
//   }
// ]
```

重要: `.eq()` 等の条件を必ず指定すること！

`.update()` に条件を付けないと **テーブルの全行が更新されてしまいます**。これは非常に危険です。必ず `.eq('id', bookId)` のような条件を付けてください。

読書状態だけを更新する例

```
// ステータスだけを変更
const { data, error } = await supabase
  .from('books')
  .update({ status: 'completed' })
  .eq('id', bookId)
  .select();
```

7.4 DELETE（データの削除）

```
// 特定の書籍を削除
const bookId = '550e8400-e29b-41d4-a716-446655440000';

const { error } = await supabase
  .from('books')
  .delete()
  .eq('id', bookId); // 必ず条件を指定すること！

if (error) {
  console.error('削除エラー:', error.message);
  return;
}

console.log('書籍を削除しました');
```

重要: `.eq()` 等の条件を必ず指定すること！

`.delete()` に条件を付けないと テーブルの全行が削除されてしまいます。復元はできません。必ず条件を付けてください。

削除されたデータを確認する

```
// 削除されたデータを返り値で確認したい場合
const { data, error } = await supabase
  .from('books')
  .delete()
  .eq('id', bookId)
  .select(); // select() を付けると削除されたデータが返る

console.log('削除されたデータ:', data);
```

7.5 エラーハンドリングのパターン

Supabase の操作では、常にエラーが発生する可能性があります。以下は推奨されるエラーハンドリングのパターンです。

```
// 汎用的なエラーハンドリング関数
async function fetchBooks() {
  const { data, error } = await supabase
    .from('books')
    .select('*')
    .order('created_at', { ascending: false });

  if (error) {
    // エラーの種類に応じた処理
    console.error('Supabase エラー:', {
      message: error.message, // エラーメッセージ
      code: error.code,       // エラーコード
      details: error.details, // 詳細情報
      hint: error.hint,       // ヒント (修正方法の提案)
    });
    throw new Error(`書籍の取得に失敗しました: ${error.message}`);
  }

  return data;
}
```

8. テストデータの投入

8.1 SQL でサンプルデータを投入

Supabase の SQL Editor で以下の SQL を実行して、テスト用のサンプルデータを5件投入します。

```

-- =====
-- テストデータの投入
-- 書籍管理アプリのサンプルデータ
-- =====

INSERT INTO books (title, author, publisher, published_date, rating, status, notes) VALUES
(
  'ノルウェイの森',
  '村上春樹',
  '講談社',
  '1987-09-04',
  5,
  'completed',
  '名作。静かで美しい文体に引き込まれた。何度も読み返したくなる作品。'
),
(
  '人間失格',
  '太宰治',
  '新潮社',
  '1948-06-25',
  4,
  'completed',
  '太宰治の代表作。人間の弱さと苦悩が痛いほど伝わってくる。'
),
(
  'こころ',
  '夏目漱石',
  '岩波書店',
  '1914-09-20',
  5,
  'reading',
  '先生の手紙の部分を読んでいる。明治時代の人間関係の複雑さが興味深い。'
),
(
  '銀河鉄道の夜',
  '宮沢賢治',
  '岩波書店',
  '1934-01-01',
  NULL,
  'want_to_read',
  '友人に勧められた。幻想的な世界観が気になる。'
),
(
  'コンビニ人間',
  '村田沙耶香',
  '文藝春秋',
  '2016-07-27',
  4,
  'completed',

```



```
'芥川賞受賞作。「普通」とは何かを考えさせられた。読みやすく一気に読了。'  
);
```

実行後、`Success. 5 rows affected.` と表示されれば成功です。

8.2 Supabase JavaScript クライアントからデータを投入する方法

SQL の代わりに、JavaScript/TypeScript コードからもテストデータを投入できます。

// テストデータの投入スクリプト（開発時のみ使用）

```
import { supabase } from '@lib/supabase';
import type { BookInsert } from '@types/book';

const sampleBooks: BookInsert[] = [
  {
    title: 'ノルウェイの森',
    author: '村上春樹',
    publisher: '講談社',
    published_date: '1987-09-04',
    rating: 5,
    status: 'completed',
    notes: '名作。静かで美しい文体に引き込まれた。何度も読み返したくなる作品。',
  },
  {
    title: '人間失格',
    author: '太宰治',
    publisher: '新潮社',
    published_date: '1948-06-25',
    rating: 4,
    status: 'completed',
    notes: '太宰治の代表作。人間の弱さと苦悩が痛いほど伝わってくる。',
  },
  {
    title: 'こころ',
    author: '夏目漱石',
    publisher: '岩波書店',
    published_date: '1914-09-20',
    rating: 5,
    status: 'reading',
    notes: '先生の手紙の部分を読んでいる。明治時代の人間関係の複雑さが興味深い。',
  },
  {
    title: '銀河鉄道の夜',
    author: '宮沢賢治',
    publisher: '岩波書店',
    published_date: '1934-01-01',
    status: 'want_to_read',
    notes: '友人に勧められた。幻想的な世界観が気になる。',
  },
  {
    title: 'コンビニ人間',
    author: '村田沙耶香',
    publisher: '文藝春秋',
    published_date: '2016-07-27',
    rating: 4,
    status: 'completed',
    notes: '芥川賞受賞作。「普通」とは何かを考えさせられた。読みやすく一気に読了。',
  },
]
```

```

    },
  ];

  async function seedBooks() {
    // 既存データを削除 (テスト環境のみ)
    const { error: deleteError } = await supabase
      .from('books')
      .delete()
      .neq('id', '00000000-0000-0000-0000-000000000000'); // 全行削除の安全策

    if (deleteError) {
      console.error('削除エラー:', deleteError.message);
      return;
    }

    // サンプルデータを挿入
    const { data, error } = await supabase
      .from('books')
      .insert(sampleBooks)
      .select();

    if (error) {
      console.error('挿入エラー:', error.message);
      return;
    }

    console.log(`${data.length}件のサンプルデータを投入しました`);
    data.forEach((book) => {
      console.log(`  - ${book.title} (${book.author})`);
    });
  }

  // 実行
  seedBooks();

```

8.3 Supabase Dashboard での確認方法

テストデータが正しく投入されたかを Supabase Dashboard で確認しましょう。

ステップ 1: Table Editor を開く

左メニューの「Table Editor」をクリックします。

ステップ 2: books テーブルを選択

テーブル一覧から **books** をクリックします。

ステップ 3: データの確認

以下のような画面が表示されるはずです:

Table Editor > books

[+ Insert row] [Filter] [Sort]

id	title	author	rating	status	notes
3f..	ノルウェイの森	村上春樹	5	compl..	名作...
7a..	人間失格	太宰治	4	compl..	太宰...
b2..	こころ	夏目漱石	5	readi..	先生...
e5..	銀河鉄道の夜	宮沢賢治	NULL	want_..	友人...
1c..	コンビニ人間	村田沙耶香	4	compl..	芥川...

Showing 5 rows

確認ポイント:

1. 5件のデータがすべて表示されていること
2. **id** が UUID 形式で自動生成されていること
3. **rating** の値が 1〜5 の範囲内であること（銀河鉄道の夜は NULL）
4. **status** の値が **reading** / **completed** / **want_to_read** のいずれかであること
5. **created_at** と **updated_at** が自動的に設定されていること

ステップ 4: SQL Editor でデータを確認（オプション）

SQL Editor で以下のクエリを実行して、データを確認することもできます:

```
-- 全件取得
SELECT * FROM books ORDER BY created_at DESC;

-- 読了済みの書籍のみ取得
SELECT title, author, rating FROM books WHERE status = 'completed' ORDER BY rating DESC;

-- 件数の確認
SELECT COUNT(*) FROM books;

-- ステータスごとの件数
SELECT status, COUNT(*) as count FROM books GROUP BY status;
```

9. トラブルシューティング

9.1 接続エラー

エラー: **FetchError: request to https://xxxxx.supabase.co/rest/v1/books failed**

原因: Supabase に接続できない。

対処法:

1. 環境変数を確認する `bash # .env.local` の内容を確認（ターミナルで実行） `cat .env.local` - `NEXT_PUBLIC_SUPABASE_URL` が正しいか確認 - `NEXT_PUBLIC_SUPABASE_ANON_KEY` が正しいか確認 - URL の末尾にスラッシュ `/` がないか確認（不要です） - 余分な空白や改行がないか確認
2. プロジェクトが起動しているか確認 - Supabase Dashboard にアクセスして、プロジェクトが「Active」状態か確認 - 一時停止されている場合は「Restore project」をクリック
3. 開発サーバーを再起動 `bash # 環境変数の変更はサーバー再起動が必要 # Ctrl + C` で停止してから `npm run dev`

エラー: **TypeError: fetch failed** または **ECONNREFUSED**

原因: ネットワーク接続の問題。

対処法:

1. インターネットに接続されているか確認
2. VPN を使用している場合は一時的に無効化してみる
3. ファイアウォールが Supabase への接続をブロックしていないか確認

9.2 RLS でデータが取得できない

症状: `data` が空配列 `[]` で返ってくる（エラーは出ない）

これは Supabase で最もよくあるハマリポイントです。

原因: RLS（Row Level Security）が有効だが、アクセスを許可するポリシーが設定されていない。

確認手順:

1. Supabase Dashboard の「Authentication」>「Policies」で books テーブルのポリシーを確認
2. ポリシーが存在するか、`USING (true)` になっているか確認

対処法:

```
-- 現在のポリシーを確認
SELECT * FROM pg_policies WHERE tablename = 'books';
```

ポリシーが存在しない場合は、セクション5の SQL を再実行してください:

```
-- RLS が有効になっているか確認
SELECT relname, relrowsecurity
FROM pg_class
WHERE relname = 'books';
-- relrowsecurity が true なら RLS が有効

-- ポリシーがない場合は追加
CREATE POLICY "Allow public read access"
ON books
FOR SELECT
USING (true);

CREATE POLICY "Allow public insert access"
ON books
FOR INSERT
WITH CHECK (true);

CREATE POLICY "Allow public update access"
ON books
FOR UPDATE
USING (true)
WITH CHECK (true);

CREATE POLICY "Allow public delete access"
ON books
FOR DELETE
USING (true);
```

もう一つの対処法（開発時のみ）:

一時的に RLS を無効化して、RLS が原因かどうかを切り分けることもできます。

```
-- ⚠ 開発環境でのみ使用すること！
ALTER TABLE books DISABLE ROW LEVEL SECURITY;
```

これでデータが取得できるようになったら、RLS のポリシー設定に問題があることが確定します。問題を解決したら必ず RLS を再度有効化してください:

```
ALTER TABLE books ENABLE ROW LEVEL SECURITY;
```

症状: INSERT はできるが SELECT でデータが返らない

原因: SELECT のポリシーが設定されていない、または条件が間違っている。

対処法: 上記の SELECT ポリシーを確認してください。

9.3 型エラー

エラー: **Property 'books' does not exist on type 'Database'**

原因: TypeScript の型定義ファイルが古いか、生成されていない。

対処法:

```
# 型定義を再生成
npm run gen:types
```

エラー: **Type 'string' is not assignable to type 'number'**

原因: Supabase から返されるデータの型と、コードで期待している型が一致していない。

対処法:

```
// 自動生成された型を使用する
import type { Book } from '@types/book';

// ✖ 手で型を定義しない
interface Book {
  id: number; // UUID は stringなのに number にしている
  // ...
}

// ✔ 自動生成された型を使う
type Book = Database['public']['Tables']['books']['Row'];
```

エラー: **Could not find a declaration file for module '@supabase/supabase-js'**

原因: `@supabase/supabase-js` がインストールされていないか、TypeScript の設定に問題がある。

対処法:

```
# パッケージを再インストール
npm install @supabase/supabase-js

# node_modules を削除して再インストール
rm -rf node_modules package-lock.json
npm install
```

9.4 SQL エラー

エラー: **relation "books" already exists**

原因: テーブルがすでに存在する状態で **CREATE TABLE** を実行した。

対処法:

```
-- テーブルを削除してから再作成 (データも消える)
DROP TABLE IF EXISTS books;

-- または、テーブルが存在しない場合のみ作成
CREATE TABLE IF NOT EXISTS books (
  -- ...
);
```

エラー: **new row violates check constraint "books_rating_check"**

原因: **rating** カラムに 1～5 の範囲外の値を挿入しようとした。

対処法:

```
// ✖ 範囲外の値
const book = { title: 'テスト', author: 'テスト', rating: 10 };

// ✔ 1～5 の範囲内の値
const book = { title: 'テスト', author: 'テスト', rating: 5 };
```

エラー: **new row violates check constraint "books_status_check"**

原因: **status** カラムに許可されていない値を挿入しようとした。

対処法:

```
// ✖ 許可されていない値
const book = { title: 'テスト', author: 'テスト', status: 'done' };

// ✔ 許可された値のいずれか
const book = { title: 'テスト', author: 'テスト', status: 'completed' };
// 許可された値: 'reading' | 'completed' | 'want_to_read'
```

9.5 よくある質問

Q: anon key が漏れたらどうなりますか？

A: anon key は「公開キー」なので、ブラウザの JavaScript から見える前提で設計されています。RLS を正しく設定していれば、anon key だけではデータに不正アクセスできません。ただし、RLS を無効にしている場合は危険です。本番環境では必

ず RLS を有効にしてください。

Q: `service_role` key はいつ使いますか？

A: `service_role` key は RLS をバイパスする管理者用のキーです。絶対にフロントエンドに含めないでください。サーバーサイドのバッチ処理やマイグレーションスクリプトでのみ使用します。

Q: テーブルの定義を変更したい場合は？

A: SQL Editor で `ALTER TABLE` コマンドを使用します。変更後は `npm run gen:types` で型定義を再生成してください。

```
-- カラムの追加
ALTER TABLE books ADD COLUMN page_count integer;

-- カラムの削除
ALTER TABLE books DROP COLUMN page_count;

-- カラムの型変更
ALTER TABLE books ALTER COLUMN rating TYPE smallint;
```

まとめ

この章で学んだことを振り返りましょう。

Chapter 5 Summary

Supabase Basics

BaaS concept
Firebase comparison
PostgreSQL benefits

DB Design

Table / Row / Column
PK / FK
Data types & constraints
books table design

Security

RLS concept
Policy configuration

Client Setup

supabase-js install
Env variables
Client initialization
TS type generation

CRUD Operations

SELECT (Read)
INSERT (Create)
UPDATE (Update)
DELETE (Delete)

この章で達成したこと:

1. Supabase アカウントを作成し、プロジェクトをセットアップした
2. リレーショナルデータベースの基礎を学んだ
3. 書籍管理アプリの **books** テーブルを設計・作成した
4. RLS を設定してセキュリティを確保した
5. Supabase クライアントをセットアップした
6. CRUD の全操作（SELECT / INSERT / UPDATE / DELETE）を学んだ
7. テストデータを投入してデータベースの動作を確認した

次の章では:

第6章では、ここで作成した Supabase データベースと React のフロントエンドを接続し、実際に動く書籍管理アプリの画面を構築していきます。